

Eigenvector Equivalence of the Two-Phase PageRank Model

From Power Iteration to the Google Matrix

Two-Phase PageRank Project

March 27, 2026

Abstract

This document rigorously derives how the Two-Phase PageRank power iteration can be reformulated as a standard eigenvector problem by absorbing the teleportation vector $\mathbf{E}_{2N}$ into the transition matrix M_{2N} , forming the *Google Matrix* $\tilde{M}$. We prove that the stationary distribution $\mathbf{v}^*$ is the unique principal eigenvector of $\tilde{M}$ with eigenvalue $\lambda = 1$, and explain why power iteration is the preferred computational method.

Contents

1	The Two-Phase PageRank Formulation	1
1.1	The $2N \times 2N$ Transition Matrix	1
1.2	The Teleportation Vector	1
1.3	The Power Iteration	2
2	Absorbing $\mathbf{E}_{2N}$ into the Matrix	2
2.1	Key Observation	2
2.2	The Google Matrix Construction	2
2.3	Derivation of Equivalence	2
3	Properties of $\tilde{M}$	3
4	Why Power Iteration is Used Instead	3
5	Convergence Rate	4
6	Visual Summary	4
7	Conclusion	4

1 The Two-Phase PageRank Formulation

1.1 The $2N \times 2N$ Transition Matrix

Let the road network have N directed links. Our Two-Phase model constructs a block matrix of dimension $2N \times 2N$:

$$M_{2N} = \begin{pmatrix} (1 - \beta) P_{\text{up}} & \beta I_N \\ \mathbf{0}_{N \times N} & P_{\text{down}} \end{pmatrix} \quad (1)$$

where:

- P_{up} is the **upstream transition matrix** with travel-time self-loops: a vehicle on link i either stays (self-loop weight $w_i = \mu \cdot \ell_i / s_i$) or moves to an upstream neighbor.
- P_{down} is the **downstream transition matrix**: uniform splitting among downstream neighbors.
- $\beta \in (0, 1)$ is the **phase-switch probability**: at each step, a vehicle in the upstream phase switches to the downstream phase with probability β .
- I_N is the $N \times N$ identity matrix (the phase-switch landing).

1.2 The Teleportation Vector

The teleportation (personalization) vector encodes the ground-truth traffic distribution derived from GPS observations:

$$\mathbf{E}_{2N} = \begin{pmatrix} \mathbf{E}_N \\ \mathbf{0}_N \end{pmatrix} \in \mathbb{R}^{2N}, \quad \sum_{i=1}^N E_N[i] = 1, \quad E_N[i] \geq 0. \quad (2)$$

Vehicles are always *teleported* (injected) into the upstream phase, never directly into the downstream phase.

1.3 The Power Iteration

The stationary distribution $\mathbf{v}^*$ is computed by iterating:

$$\boxed{\mathbf{v}^{(t+1)} = d \cdot M_{2N}^T \mathbf{v}^{(t)} + (1 - d) \cdot \mathbf{E}_{2N}} \quad (3)$$

where $d \in (0, 1)$ is the **damping factor** (we use $d = 0.80$). At convergence, $\mathbf{v}^* = \lim_{t \rightarrow \infty} \mathbf{v}^{(t)}$.

2 Absorbing $\mathbf{E}_{2N}$ into the Matrix

2.1 Key Observation

The teleportation term $(1 - d) \mathbf{E}_{2N}$ does not depend on $\mathbf{v}$. It is a constant additive correction applied at every iteration. We can eliminate it by constructing a single matrix that encodes both the transition dynamics *and* the teleportation.

2.2 The Google Matrix Construction

Definition 1 (Google Matrix). Define the **Google Matrix** $\widetilde{M} \in \mathbb{R}^{2N \times 2N}$ as:

$$\boxed{\widetilde{M} = d \cdot M_{2N}^T + (1 - d) \cdot \mathbf{E}_{2N} \mathbf{1}^T} \quad (4)$$

where $\mathbf{1} \in \mathbb{R}^{2N}$ is the all-ones vector.

Remark 1. The term $\mathbf{E}_{2N} \mathbf{1}^T$ is a **rank-1 matrix** of size $2N \times 2N$. Its (i, j) -th entry is simply $E_{2N}[i]$, regardless of j . This means that from *every* state j , there is a probability $(1 - d) \cdot E_{2N}[i]$ of teleporting to state i .

2.3 Derivation of Equivalence

Theorem 1 (Eigenvector Equivalence). The fixed point of the power iteration (3) is the principal eigenvector of $\widetilde{M}$ with eigenvalue $\lambda = 1$:

$$\mathbf{v}^* = \widetilde{M} \mathbf{v}^*.$$

Proof. At convergence, $\mathbf{v}^{(t+1)} = \mathbf{v}^{(t)} = \mathbf{v}^*$. Substituting into (3):

$$\mathbf{v}^* = d \cdot M_{2N}^T \mathbf{v}^* + (1 - d) \cdot \mathbf{E}_{2N}. \quad (5)$$

Since $\mathbf{v}^*$ is a probability vector, $\mathbf{1}^T \mathbf{v}^* = 1$. Therefore:

$$\begin{aligned} (1 - d) \cdot \mathbf{E}_{2N} &= (1 - d) \cdot \mathbf{E}_{2N} \cdot \mathbf{1} \\ &= (1 - d) \cdot \mathbf{E}_{2N} \cdot (\mathbf{1}^T \mathbf{v}^*) \\ &= [(1 - d) \cdot \mathbf{E}_{2N} \mathbf{1}^T] \mathbf{v}^*. \end{aligned} \quad (6)$$

Substituting (6) into (5):

$$\begin{aligned} \mathbf{v}^* &= d \cdot M_{2N}^T \mathbf{v}^* + [(1 - d) \cdot \mathbf{E}_{2N} \mathbf{1}^T] \mathbf{v}^* \\ &= \underbrace{[d \cdot M_{2N}^T + (1 - d) \cdot \mathbf{E}_{2N} \mathbf{1}^T]}_{\widetilde{M}} \mathbf{v}^* \\ &= \widetilde{M} \mathbf{v}^*. \end{aligned} \quad \square$$

3 Properties of $\widetilde{M}$

Proposition 1 ($\widetilde{M}$ is Column-Stochastic). Every column of $\widetilde{M}$ sums to 1.

Proof. For any column j :

$$\begin{aligned} \mathbf{1}^T \widetilde{M}_{:,j} &= d \cdot \mathbf{1}^T (M_{2N}^T)_{:,j} + (1 - d) \cdot \mathbf{1}^T (\mathbf{E}_{2N} \mathbf{1}^T)_{:,j} \\ &= d \cdot \underbrace{\sum_i M_{2N}[j, i]}_{=1 \text{ (row-stochastic)}} + (1 - d) \cdot \underbrace{\sum_i E_{2N}[i]}_{=1} \\ &= d + (1 - d) = 1. \end{aligned} \quad \square$$

Proposition 2 (Unique Stationary Distribution). $\widetilde{M}$ has a unique eigenvector with eigenvalue $\lambda = 1$.

Proof. The rank-1 perturbation $(1 - d) \mathbf{E}_{2N} \mathbf{1}^T$ has all strictly positive entries (since $E_N[i] > 0$ for all i after Laplace smoothing and $1 - d > 0$). Therefore $\widetilde{M}$ is a **primitive** stochastic matrix (every entry is strictly positive in at least one power). By the **Perron–Frobenius theorem**, a primitive stochastic matrix has a unique stationary distribution, corresponding to the unique eigenvector with eigenvalue $\lambda = 1$. $\square$

4 Why Power Iteration is Used Instead

Although $\mathbf{v}^*$ is theoretically the principal eigenvector of $\widetilde{M}$, computing it via eigendecomposition is impractical:

Property	Eigendecomposition	Power Iteration
Matrix form	Dense $\widetilde{M}$	Sparse M_{2N} + vector $\mathbf{E}_{2N}$
Memory	$O(4N^2)$ entries	$O(\text{nnz}(M_{2N}) + 2N)$
Time complexity	$O((2N)^3)$	$O(k \cdot \text{nnz}(M_{2N}))$
For $N = 99,716$:		
Memory	≈ 300 GB	≈ 50 MB
Time	$\approx$ years	≈ 3 seconds
Iterations	1 (but cubic)	$k \approx 40$

Key Insight

Power iteration **is** the eigenvector computation—it is simply the most efficient algorithm for finding the principal eigenvector of a matrix that is the sum of a large sparse matrix and a small rank-1 correction. Each iteration computes:

$$\mathbf{v}^{(t+1)} = \underbrace{d \cdot M_{2N}^T \mathbf{v}^{(t)}}_{\text{sparse matvec: } O(\text{nnz})} + \underbrace{(1-d) \cdot \mathbf{E}_{2N}}_{\text{constant vector add: } O(2N)}$$

without ever forming the dense $\widetilde{M}$ explicitly.

5 Convergence Rate

The convergence rate of power iteration on $\widetilde{M}$ is governed by the ratio of the two largest eigenvalues. For the Google Matrix:

Theorem 2 (Convergence Rate). The power iteration converges geometrically with rate:

$$\|\mathbf{v}^{(t)} - \mathbf{v}^*\| \leq C \cdot d^t$$

where d is the damping factor. The second-largest eigenvalue of $\widetilde{M}$ satisfies $|\lambda_2| \leq d$.

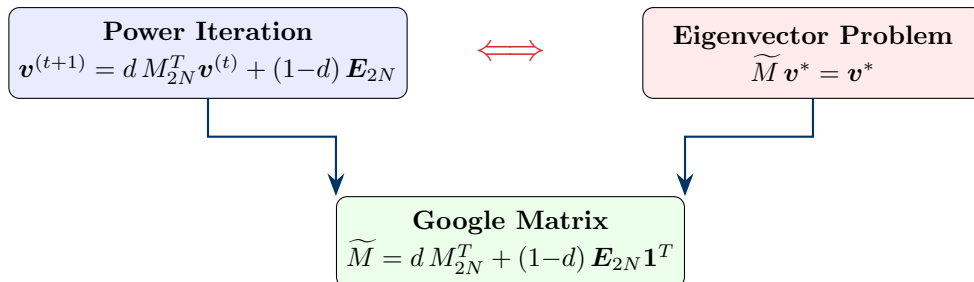
For our parameters ($d = 0.80$), this means:

- After 10 iterations: error $\leq C \cdot 0.8^{10} \approx 0.107 C$
- After 20 iterations: error $\leq C \cdot 0.8^{20} \approx 0.012 C$
- After 40 iterations: error $\leq C \cdot 0.8^{40} \approx 0.0001 C$

This explains why our implementation consistently converges in ~ 37 – 42 iterations with tolerance 10^{-6} .

6 Visual Summary

The following diagram summarizes the two equivalent viewpoints:



7 Conclusion

The Two-Phase PageRank model admits two mathematically equivalent formulations:

1. **Iterative:** Repeatedly apply $\mathbf{v} \leftarrow d M_{2N}^T \mathbf{v} + (1 - d) \mathbf{E}_{2N}$ until convergence.
2. **Spectral:** Find the principal eigenvector (eigenvalue $\lambda = 1$) of the Google Matrix $\widetilde{M} = d M_{2N}^T + (1 - d) \mathbf{E}_{2N} \mathbf{1}^T$.

Both yield *exactly the same* stationary distribution $\mathbf{v}^*$. The power iteration is preferred because it exploits the sparsity of M_{2N} and never materializes the dense Google Matrix, reducing memory from ~ 300 GB to ~ 50 MB and computation time from years to seconds.